Solutions: Zero-Downtime Releases at ShopKart

Nov 3, 2026 · @Utkarsh Agarwal

This is the worked solution to the ShopKart assignment. Try each part yourself first, then compare: every manifest runs as written on Docker Desktop, and each step explains why it works.

Before you start

- Pre-pull images so the college network does not slow the lab: `docker pull nginx:1.25 && docker pull nginx:1.26 && docker pull curlimages/curl`.
- Check the context: `kubectl config use-context docker-desktop`, then `kubectl get nodes`.
- Create the helper Pod once and reuse it between parts:

```
kubectl run tester --image=curlimages/curl --restart=Never -- sleep 3600
kubectl wait --for=condition=Ready pod/tester
```

- `kubectl delete all --all` also deletes `tester`, because it is a plain Pod. Re-create it after each clean-up.

Reusable test commands

Purpose	Command
Status loop (twice a second)	`kubectl exec tester -- sh -c "while true; do curl -s -o /dev/null -w '%{http_code}\n' --max-time 1 <svc>; sleep 0.5; done"`
Which version answered	`kubectl exec tester -- curl -sI --max-time 2 <svc> | grep -i ^server`
Count versions over 100 requests	`kubectl exec tester -- sh -c 'for i in $(seq 1 100); do curl -sI --max-time 2 <svc> | grep -i ^server; done' | sort | uniq -c`

Purpose	Command
Pod watcher	`kubectl get pods -w`

The `Server` header is the trick that makes versions visible: nginx 1.25 answers with `nginx/1.25.x` and nginx 1.26 with `nginx/1.26.x`, so no custom image is needed.

Note on the timing of the loops

Nginx stops the instant it is told to, and the Service needs a moment to stop sending it requests. One or two isolated `000` lines during a rolling update are normal and are not a failure. A run of five or more consecutive `000` lines is real downtime. Judge your results by the length of the gap, not by the presence of a single failure.

Part 1 solution: the safe rolling update

The key idea: `maxUnavailable: 0` plus a readiness probe means a new Pod must be proven healthy before any old Pod is removed, so capacity never drops below 4.

The manifest (`shopkart.yaml`)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shopkart
  labels:
    app: shopkart
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: shopkart
  template:
    metadata:
      labels:
```

```
        app: shopkart
    spec:
      containers:
      - name: web
        image: nginx:1.25
        ports:
        - containerPort: 80
        readinessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 3
          periodSeconds: 5
        resources:
          requests:
            cpu: 50m
            memory: 64Mi
          limits:
            cpu: 200m
            memory: 128Mi
---
apiVersion: v1
kind: Service
metadata:
  name: shopkart-svc
spec:
  type: ClusterIP
  selector:
    app: shopkart
  ports:
  - port: 80
    targetPort: 80
```

Why each part is there

- `selector.matchLabels` and `template.metadata.labels` must agree (`app: shopkart`). This is how the Deployment finds its own Pods; a mismatch makes `kubectl apply` fail.
- The container is named `web`. That name matters later, because it is what you would use in `kubectl set image deployment/shopkart web=...`.

- `maxSurge: 1` lets Kubernetes run a 5th Pod during the update. `maxUnavailable: 0` forbids dropping below 4 ready Pods.
- The readiness probe makes the Service ignore a Pod until it answers on `/`. Without it, a Pod that has only just started could receive traffic and fail.
- The Service selects `app: shopkart`, not Pod names or IPs, so it keeps working as Pods are replaced.

Commands, in order

```
kubectl apply -f shopkart.yaml --dry-run=client
kubectl apply -f shopkart.yaml
kubectl get deploy,pods -l app=shopkart
kubectl get endpoints shopkart-svc
```

Expected: `READY 4/4`, and the endpoints line lists 4 `IP:80` addresses. If it shows `<none>`, compare the Service selector with `kubectl get pods --show-labels`.

Start the watcher in one terminal and the status loop in another (see the helper commands above). Then change `image: nginx:1.25` to `nginx:1.26` in the YAML and add this under `metadata:`:

```
  annotations:
    kubernetes.io/change-cause: "upgrade to nginx 1.26"
```

```
kubectl apply -f shopkart.yaml
kubectl rollout status deployment/shopkart
kubectl rollout history deployment/shopkart
kubectl get rs -l app=shopkart
```

What you should see

- In the watcher, a new Pod goes `ContainerCreating`, then `Running` with `READY 0/1`, then `1/1`. Only after that does an old Pod go `Terminating`. This repeats four times, with never fewer than 4 ready Pods.
- The status loop prints `200` throughout. One or two isolated `000` lines are possible and are not downtime.
- `kubectl get rs` shows two ReplicaSets: the old one at `DESIRED 0` and the new one at `DESIRED 4`. The old one is kept so that rollback is instant.

- `kubectl rollout history` shows two revisions, with your change-cause text on the second.

Written answer (model)

The loop stayed at 200 because Kubernetes added one new Pod at a time (`maxSurge: 1`) and only removed an old Pod once the new one passed its readiness probe (`maxUnavailable: 0`). The Service sent traffic only to ready Pods, so there was always a full set serving. With `Recreate`, all four old Pods would be stopped before any new one started, so the loop would show a run of `000` for several seconds until the new Pods became ready.

Part 2 solution: blue/green

The key idea: run both versions at full size side by side, and let one Service selector decide which one customers reach. Switching versions means changing one label value, not replacing any Pods.

Manifests

`blue.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shopkart-blue
  labels:
    app: shopkart
    version: blue
spec:
  replicas: 3
  selector:
    matchLabels:
      app: shopkart
      version: blue
  template:
    metadata:
      labels:
        app: shopkart
        version: blue
    spec:
```

```
      containers:
      - name: web
        image: nginx:1.25
        ports:
        - containerPort: 80
        readinessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 3
          periodSeconds: 5
```

`green.yaml` is identical except for three things: `name: shopkart-green`, every `version: blue` becomes `version: green`, and `image: nginx:1.26`.

`service.yaml` (the live Service, pointing at blue first):

```
apiVersion: v1
kind: Service
metadata:
  name: shopkart-live
spec:
  type: NodePort
  selector:
    app: shopkart
    version: blue
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
```

`preview-service.yaml` (a private door to green):

```
apiVersion: v1
kind: Service
metadata:
  name: shopkart-preview
spec:
  type: NodePort
  selector:
    app: shopkart
    version: green
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30081
```

Steps and what to expect

```
kubectl apply -f blue.yaml -f service.yaml
kubectl exec tester -- curl -sI --max-time 2 shopkart-live | grep -i ^server
```

The header shows `nginx/1.25.x`.

```
kubectl apply -f green.yaml -f preview-service.yaml
kubectl exec tester -- curl -sI --max-time 2 shopkart-live    | grep -i
^server
kubectl exec tester -- curl -sI --max-time 2 shopkart-preview | grep -i
^server
kubectl get endpoints shopkart-live shopkart-preview -o wide
```

The first command still prints 1.25 (customers see blue) and the second prints 1.26 (you are testing green). The endpoint lists show 3 different Pod IPs each. This is the whole point of blue/green: the new version is running on real infrastructure and is testable, yet no customer touches it.

The switch. Start the status/Server loop against `shopkart-live`, then edit `service.yaml` and change `version: blue` to `version: green`:

```
kubectl apply -f service.yaml
```

The loop flips from 1.25 to 1.26 within a second or two, with no run of `000`. `kubectl get pods` is unchanged: no Pod was created or deleted. Only the Service's endpoint list changed.

The rollback. Change the selector back to `version: blue` and apply again. The loop flips back to 1.25 just as fast. Compare that with `kubectl rollout undo`, which has to scale ReplicaSets up and wait for Pods to become ready: here blue was never turned off, so there is nothing to wait for.

Clean-up after sign-off. Set `replicas: 0` in `blue.yaml` and apply. Keep the file: next release, blue becomes the idle environment you deploy to. Keep blue running until you are confident, because that is your instant rollback.

Written answers (model)

- **Why the switch is instant and needs no rollout:** the Service picks Pods by label, and green's Pods were already running and ready. Changing the selector only changes which Pod IPs are in the endpoint list; no Pods start, stop or restart.
- **What it costs:** you pay for double the Pods (6 instead of 3) while both environments exist. In step 5 both versions are fully running, so the cluster needs capacity for both. A rolling update only needs one spare Pod.
- **When it is clearly better:** a high-risk release you want to fully test before exposing it, or when you need rollback in seconds. Example: a checkout redesign during a sales event, where every minute of a broken page costs money.

Part 3 solution: canary

The key idea: one Service selects Pods from two Deployments, and the share of traffic each version gets is roughly its share of the Pods. 9 stable Pods plus 1 canary Pod means about 10% of requests reach the canary.

Manifests

`stable.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shopkart-stable
  labels:
    app: shopkart
    track: stable
spec:
  replicas: 9
```

```
  selector:
    matchLabels:
      app: shopkart
      track: stable
  template:
    metadata:
      labels:
        app: shopkart
        track: stable
    spec:
      containers:
      - name: web
        image: nginx:1.25
        ports:
        - containerPort: 80
        readinessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 3
          periodSeconds: 5
        resources:
          requests:
            cpu: 25m
            memory: 32Mi
```

`canary.yaml` is the same, except `name: shopkart-canary`, `replicas: 1`, every `track: stable` becomes `track: canary`, and `image: nginx:1.26`.

`service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: shopkart-svc
spec:
  selector:
    app: shopkart
  ports:
  - port: 80
    targetPort: 80
```

Small resource requests (25m CPU, 32Mi memory) matter here: 10 Pods must fit on the single Docker Desktop node. If Pods stay `Pending`, run `kubectl describe pod <name>` and read the Events, then lower the requests or use 3 stable and 1 canary.

Why the selectors are built this way

- The **Service** selects only `app: shopkart`, so it matches Pods from both Deployments.
- Each **Deployment** selects `app` and `track`. If both Deployments selected only `app: shopkart`, each would think the other's Pods were its own, and they would fight over the replica count.

Measuring the split

```
kubectl apply -f stable.yaml -f canary.yaml -f service.yaml
kubectl get endpoints shopkart-svc
kubectl exec tester -- sh -c 'for i in $(seq 1 100); do curl -sI --max-time 2
shopkart-svc | grep -i ^server; done' | sort | uniq -c
```

The endpoints line lists 10 IPs. A typical result looks like this, but your numbers will differ:

Version	Run 1	Run 2	Run 3
nginx/1.25	88	93	91
nginx/1.26	12	7	9

The split is close to 90/10 but never exact, because kube-proxy picks a Pod at random for each new connection. Treat 5 to 15 canary hits per 100 as correct.

Promoting in stages

Edit `replicas:` in both files and apply, then re-test each time:

Stage	Stable	Canary	Expected canary share
1	9	1	about 10%
2	7	3	about 30%
3	5	5	about 50%
4	0	10	100%

Finish by changing `stable.yaml` to `image: nginx:1.26` and `replicas: 10`, applying it (this is a normal rolling update of the stable Deployment), then deleting the canary:

```
kubectl apply -f stable.yaml
kubectl rollout status deployment/shopkart-stable
kubectl delete -f canary.yaml
```

Re-run the 100-request test: all 100 answers should now be `nginx/1.26`.

The abort path: a broken canary

Change `canary.yaml` to `image: nginx:9.99` and apply:

```
kubectl get pods -l track=canary
kubectl describe pod <canary-pod-name>
kubectl get endpoints shopkart-svc
```

The canary Pod shows `ErrImagePull`, then `ImagePullBackOff`, and the Events say the manifest for `nginx:9.99` was not found. The endpoint list still shows only the 9 stable Pods, so the 100-request test returns 100 answers from nginx 1.25. Customers are unaffected because a Pod that is not Ready is never added to the Service endpoints.

The abort action is simply `kubectl delete -f canary.yaml` (or setting its replicas to 0). Stable was never touched.

Written answers (model)

- **Why replica counts control the split:** the Service spreads requests roughly evenly across all Ready Pods it matches. With 10 Pods and 1 on the new version, about 1 in 10 requests lands on it. Changing replicas changes that ratio.

- **Smallest share with 4 Pods:** 1 canary and 3 stable gives 25%. This method cannot go below 1 divided by the total number of Pods, and it cannot target specific users or send an exact 1%. Tools such as ingress controllers or service meshes split traffic by percentage or header instead.
- **Why the readiness probe matters on the canary:** with a probe, a canary that starts but cannot serve is kept out of the endpoints, so users never reach it. Without a probe, Kubernetes treats a running container as Ready, so a broken canary would receive its share of real traffic and return errors to about 10% of users.

Part 4 solution: the incident and strategy review

The key idea: a bad release should stall, not take the site down. Whether it stalls or breaks everything depends on the strategy and the readiness probe.

Setup note

The incident targets `shopkart-stable` from Part 3. Make sure `stable.yaml` has an explicit strategy block, otherwise the default (`maxUnavailable: 25%`) applies and the result is less clear-cut. Add this under `spec:` and apply, with the image on `nginx:1.26` and 4 replicas:

```
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

Start the status loop against `shopkart-svc` and a watcher with `kubectl get pods -l track=stable -w`.

Step 1: the bad release

```
kubectl set image deployment/shopkart-stable web=nginx:9.99
kubectl rollout status deployment/shopkart-stable --timeout=30s
kubectl get pods -l track=stable
```

- `rollout status` ends with `error: timed out waiting for the condition`.
- One new Pod is stuck in `ErrImagePull` / `ImagePullBackOff`; the 4 old Pods are still `Running` and `1/1`.

- The status loop keeps printing `200`.

Why: the new Pod never becomes Ready, so it receives no traffic, and `maxUnavailable: 0` means Kubernetes was not allowed to remove an old Pod to make room. The rollout simply waits.

Step 2: recover

```
kubectl rollout undo deployment/shopkart-stable
kubectl rollout status deployment/shopkart-stable
kubectl rollout history deployment/shopkart-stable
kubectl rollout history deployment/shopkart-stable --revision=<n>
```

The stuck Pod disappears and the 4 healthy Pods remain. The undo is recorded as a new revision number, so the revision with `nginx:1.26` is the highest one. Prove which revision is running by reading the `Image:` line in `--revision=<n>` for the highest revision, or with `kubectl describe deployment shopkart-stable`.

Step 3: the same mistake with Recreate

Replace the strategy block with:

```
  strategy:
    type: Recreate
```

The `rollingUpdate:` lines must be removed, because `maxSurge` and `maxUnavailable` are rejected with `Recreate`. Apply, wait for 4 Pods to be Ready, restart the loop, then repeat the bad update (`set image ... nginx:9.99`).

Now all four old Pods are terminated first, and the new Pods cannot start. The loop prints `000` continuously, and it keeps doing so until you run `kubectl rollout undo`. At two requests per second, count the `000` lines and divide by 2 to get seconds. The downtime is however long it takes you to notice and undo, typically tens of seconds in the lab, and it would be minutes in real life.

This is the whole lesson of Part 4: with `Recreate`, the bad release is the outage. With `RollingUpdate`, `maxUnavailable: 0` and a readiness probe, the bad release only stalls.

Decision table (model answer)

	Rolling update	Blue/green	Canary
How traffic moves to the new version	Pod by Pod, as new Pods become Ready	All at once, by changing the Service selector	Gradually, by changing the replica ratio
Extra capacity needed	Small (`maxSurge` extra Pods)	Double: both versions run at full size	Small: one or a few canary Pods
Rollback speed and method	Minutes: `kubectl rollout undo` re-creates Pods	Seconds: switch the selector back	Seconds: delete the canary Deployment
Two versions live together?	Yes, briefly	Yes, but only one gets customer traffic	Yes, for as long as you choose
Best use case	Routine low-risk updates of stateless apps	High-risk releases that must be tested first, instant rollback	Unknown impact; learn from a small share of real users

Recommendation (model answer)

(a) Security patch today: rolling update. It is low risk, needs no extra environment, and has zero downtime. Use `maxUnavailable: 0` and a readiness probe so a bad patch stalls instead of breaking the site. To stop it, run `kubectl rollout undo`.

(b) Database schema change the old version cannot read: Recreate. Old and new versions must never run together, because the old app would fail against the new schema. Accept a short planned downtime, schedule it in a quiet window, and take a database backup first. Rollback means restoring the backup and redeploying the old version, so test the migration beforehand.

(c) New pricing algorithm with unknown revenue impact: canary. Send about 10% of traffic to it and compare revenue or error metrics against stable before widening. To stop it, delete the canary Deployment and 100% of traffic returns to stable at once. A rolling update would expose every user within minutes.

Common mistakes and how to fix them

If your result does not match the solution above, check this table first.

Symptom	Likely cause	How to check and fix
`kubectl apply` fails with a selector error	`selector.matchLabels` does not match `template.metadata.labels`	Compare both blocks in the YAML; they must contain the same labels
Service endpoints show `<none>`	Service selector matches no Pod labels	`kubectl get pods --show-labels`, then fix the selector
Blue/green: switching the selector changes nothing	Only one label in the Service selector, or the label value has a typo	`kubectl describe svc shopkart-live` and read the `Selector:` line
Canary: Pod counts are wrong or keep changing	Both Deployments select only `app: shopkart`, so they fight over each other's Pods	Add `track: stable` / `track: canary` to each Deployment's `selector.matchLabels` and Pod labels, then delete and re-apply both
Canary split is far from 90/10	Too few requests, or the endpoint list is not 10	Run 200 requests and check `kubectl get endpoints shopkart-svc`
Loop shows a long run of `000` during a rolling update	Probe or strategy missing, or the update started before all Pods were Ready	`kubectl describe deployment <name>` should show `RollingUpdate`, `0 max unavailable` and a `Readiness:` line; wait for all Pods `1/1` before starting
`Recreate` apply is rejected	`rollingUpdate:` block left in the YAML	Remove `maxSurge` and `maxUnavailable` when the type is `Recreate`
`kubectl exec tester` says the Pod is not found	`kubectl delete all --all` removed it	Re-create it with the `kubectl run tester ...` command
Pods stay `Pending`	Not enough CPU or memory on the single node	`kubectl describe pod <name>`, read Events, lower requests or replicas

Symptom	Likely cause	How to check and fix
`localhost:30080` does not respond	NodePort not reachable on some Docker Desktop setups	Use `tester` against the Service name, or `kubectl port-forward svc/shopkart-live 8080:80`
`ImagePullBackOff` on a good image	Typo in the tag, or slow network	`kubectl describe pod <name>` ; pre-pull with `docker pull`